TradeLens

An AI-powered trading journal for Indian retail traders

Import your broker tradebook, see your true P&L after every Indian charge, find the habits that cost you money, and get a process coach that explains your own numbers and never gives tips.

Live demo	https://tradelens-roan.vercel.app
Source code	https://github.com/26sarth-sketch/tradelens
Built for	College hackathon (2-day build)
Stack	Next.js 16, TypeScript, Tailwind CSS v4, Prisma, PostgreSQL (Neon), Vercel, Google Gemini via OpenRouter
Document	Basic project documentation, 22 October 2026

Educational journaling tool. Not investment advice. TradeLens reviews a trader's own trades and habits only; it does not recommend what to buy or sell, give price targets or predict markets (that requires SEBI registration). TradeLens is an original project and is not affiliated with any other journaling product.

1. Overview

The problem. Most retail F&O traders in India lose money and cannot see why. Broker apps show raw P&L, but not how much brokerage, STT, exchange fees and GST ate, nor which habits (revenge trades, overtrading, trading without a plan, expiry days) caused the losses.

The solution. TradeLens turns a broker tradebook into a journal. It computes true net P&L with Indian charges, shows where money leaks, tracks discipline against the trader's own rules, and adds an AI coach whose every number is computed in code and verified against the data.

What makes it different

- **Built for India:** NSE/BSE, lot sizes, weekly expiry days, IST times, rupee formatting (1,02,656), all charges per trade.
- **Honest AI:** the model never does the maths. Figures come from code and each AI answer shows a "figures verified" check.
- **Process, not tips:** a compliance guardrail is in every AI prompt; tip requests are politely declined with a SEBI note.
- **Works without AI:** imports, charges, reports, risk rules, auto-tagger and a computed "what your data says" all run with no key.

2. Features

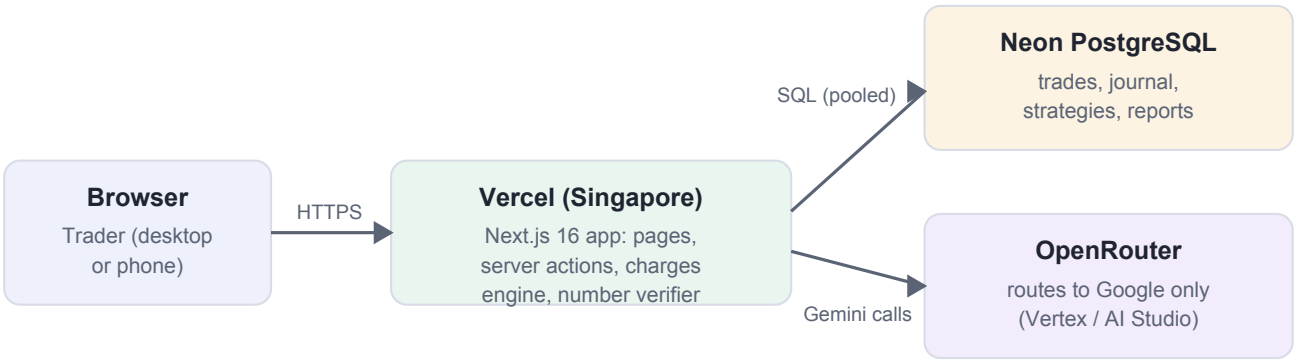
Everything below is live in the demo. Imports, charges, reports, risk rules and the auto-tagger work without an AI key.

Area	What it does
Import	Zerodha tradebook in one click; Upstox, Angel One, Groww, Dhan and others through a column mapper with presets. Fills become round-trip trades; duplicates skipped; open positions flagged; imports can be undone. Manual entry with a live charges preview.
Charges engine	Brokerage, STT, exchange charges, SEBI fee, stamp duty, GST (and DP charge on delivery sells). All rates, lot sizes and expiry rules live in `config/market.ts`.
Dashboard	Net P&L, win rate, profit factor, expectancy, max drawdown, charges as a share of gross profit. Views in ₹, % of capital, R-multiple or points. Equity curve, P&L calendar, risk widget, Lens Score radar, privacy mode.
Journal and Trade Log	Day cards with notes; filterable log; per-trade review: strategy checklist, emotions, followed-plan flag, stop-loss/target (R), high/low (money left on the table), notes, screenshot.
Reports	By day of week, time of day, segment, symbol, strategy, tag, emotion and expiry day, with a shared filter bar. Compare any two sets side by side.
Strategies	Playbooks as entry/exit/risk checklists, per-strategy stats and R distribution, rule adherence, missed-trades log.
Risk rules	Max daily loss, trades per day, loss per trade, stop after N losses in a row, no trades after a cut-off. Live status for today plus breach history.
Notebook	Pre-market plan, automatic plan-vs-actual check, end-of-day reflection with a discipline rating.
AI Coach	Ask-your-journal chat, session review, weekly review, journal analysis and feedback on each trade. Beginner/Intermediate/Expert style; English or Hinglish.

Area	What it does
Auto-tagger	Rule-based detection of revenge trades, overtrading, losers held too long and late entries, each with its reason. Accept or dismiss. No AI needed.
Lens Score and Progress	A 0-100 score from six components (win rate, profit factor, avg win/loss, drawdown, green days, plan adherence), rolling trend, disciplined-day streaks.
Learn	Eight short lessons (R-multiple, profit factor, charges in F&O, position sizing, revenge trading, expiry days, Lens Score, using the journal), linked from dashboard tooltips.
Landing page	Hero with live numbers from the demo journal, features, AI coach, INR pricing (Free / Pro ₹299 / Elite ₹699, display only), FAQ.

3. Architecture

A single Next.js application (App Router, server components and server actions) serves every page. There is no separate API server: pages read the database directly on the server and forms post to server actions.



Layer	Choice	Notes
App	Next.js 16, React, TypeScript	App Router; all journal pages are rendered on demand; AI pages allow up to 300 s per request.
UI	Tailwind CSS v4, Recharts, lucide-react	Dark by default; mobile-first (checked at 375 px); privacy mode blurs every rupee amount.
Data	PostgreSQL on Neon, Prisma 6	Pooled connection for the app; branches: **main** (live demo) and **dev** (local work).
Validation	Zod	Forms, URL filters and every AI response are validated; impossible dates are rejected.
AI	Google Gemini via OpenRouter	Default `google/gemini-3.7-flash`; structured JSON outputs and tool calling.
Hosting	Vercel	Auto-deploys on every push to `main`; functions in Singapore, next to the database.

4. How the AI stays honest

- **Numbers come from code.** Stats are computed from the trades and passed to the model; for chat, the model calls a `query_journal` tool that runs in code, so it never counts or adds anything itself.

- **Every figure is checked.** After each answer, every rupee amount, percentage and R value is compared with the data (at the precision written, or as a difference between sibling values). The card shows "All N figures verified" or lists any that could not be.
- **Compliance guardrail.** Every system prompt says: process and behaviour coaching only; no buy/sell/hold calls, targets, levels or predictions. Asked for a tip, the coach declines, mentions SEBI registration and offers a review of the trader's own history instead.
- **Google models only.** Only `google/` model IDs are accepted. Each request is pinned to Google's own infrastructure (Vertex / AI Studio) with fallbacks off. The key lives only in the `OPENROUTER_API_KEY` environment variable.
- **Resilient and visible cost.** Rate limits and server errors retry up to three times with backoff; out-of-credits stops with a clear message. Each report shows its model and cost. Reports are cached, so reloading does not call the AI again.

5. Run it locally

Requires Node.js 20+ and a free Neon PostgreSQL database.

- Create a project at neon.tech and copy its connection string (a separate Neon *branch* for local work keeps the live data safe).
- Run `npm install`. Put the connection string in `.env` as `DATABASE_URL` (the first `npm run dev` creates `.env` from `.env.example`).
- Run `npm run dev` and open http://localhost:3000. The first run creates the tables and loads about 60 trading days of demo data.
- Optional AI: `npm run set-key` (hidden prompt), restart, then `npm run ai:check` for a one-call test.

Variable	Required	Purpose
DATABASE_URL	Yes	Neon PostgreSQL connection string (pooled).
OPENROUTER_API_KEY	No	Turns on the AI Coach. Without it everything else still works.
OPENROUTER_MODEL	No	Any google/ model; default google/gemini-3.7-flash.
BLOB_READ_WRITE_TOKEN	No	Private Vercel Blob store; enables screenshot upload on Vercel (otherwise files go to uploads/ locally).

Script	What it does
npm run dev / build / start	Develop, build and serve the app.
npm run db:seed	Wipe and reload the demo data.
npm run set-key / ai:check	Save the OpenRouter key; run one test call that prints the reply, model and cost.
npm test	117 unit tests: charges, imports, stats, reports, rules, score, filters, AI number verifier.
npm run typecheck / lint	TypeScript and ESLint.

6. Deployment

The live site runs on **Vercel** with its database on **Neon**. Pushing to `main` on GitHub deploys automatically.

- **Neon:** one project, Singapore region. Tables are created once with `npx prisma db push` and the demo journal is loaded with `npm run db:seed`.

- **Vercel:** environment variables `DATABASE_URL` (secret), `OPENROUTER_MODEL` and, to enable AI, `OPENROUTER_API_KEY` (secret). The build skips database setup on Vercel.
- **Screenshots** are switched off on the hosted site unless a Blob store is connected.
- **Reset to demo data** (Settings) reloads the demo journal in about 3 seconds, on the live site too.

7. Two-minute demo

Time	Show	Say
0:00	Landing page	Most Indian F&O traders lose money and cannot see why; TradeLens is a journal built for India. Point at the live hero numbers.
0:15	Add trades, Import	Try the sample Zerodha file: fills become trades with every Indian charge; duplicates skipped. Import.
0:35	Dashboard	"Charges ate X% of gross profit"; switch to R-multiple; today's risk widget; red expiry days in the calendar; privacy mode.
0:55	Reports, Compare	Followed plan vs broke plan: the curves split. Before vs after 2 PM.
1:15	AI Coach	Ask "What's my win rate on expiry days?" and point at the verified badge; ask which stock to buy and watch it decline.
1:35	Session review	This morning's plan against what actually happened: rules broken, what went well, tomorrow's focus.
1:45	Progress	Lens Score rising as the trader kept journaling; discipline calendar turning green.
1:55	Pricing	Free / Pro ₹299 / Elite ₹699; broker auto-sync is next.

The demo journal covers 60 trading days: 193 trades, gross ₹31,464, charges ₹18,121 (58% of gross), net ₹13,343, Lens Score rising from 47 to 67. The demo data is generated by a deterministic seed.

8. Quality and testing

- **117 automated tests** cover the charges engine, importers, stats, reports, risk rules, Lens Score, filters and the AI number verifier.
- **A written release checklist** (`DEPLOY_CHECKLIST.md`) was walked end to end: inputs, outputs, errors, no-API-key mode, AI features, GitHub and Vercel. It found and fixed real bugs (for example, impossible dates in URLs and rupee amounts leaking through privacy mode).
- **Error handling:** bad CSVs, oversized or fake screenshots, wrong or missing API key, no credits, offline AI, empty journal and invalid URLs all show clear messages instead of crashing.
- **Design decisions** are recorded in `DECISIONS.md` (103 entries).

9. Known limitations and roadmap

- **No user accounts yet:** the app is a single-trader demo. Anyone with the link can edit data; Reset to demo data restores it.
- **Verify charge rates before relying on net P&L.** Rates, lot sizes and expiry days are in `config/market.ts` with a last-updated date; check them against current broker and exchange circulars.
- **Broker column presets** for Upstox, Angel One, Groww and Dhan are best guesses; confirm against a real export. Commodity and currency trades are not supported yet.

- **Screenshots** are off on the hosted demo (a Blob store is optional).
- **Roadmap:** broker API auto-sync (Kite Connect, Upstox, Dhan) through the `BrokerAdapter` interface, user accounts, and paid plans (pricing is display-only today).

10. Project layout

```
app/             Next.js pages and server actions (app/(app)/... = the journal)
components/      UI: charts, forms, coach cards, layout
config/          brand.ts, market.ts (all charge rates, lot sizes, expiry rules)
lib/             charges, stats, reports, rules, score, importers, filters
lib/ai/          OpenRouter client, guardrail, prompts, schemas, number verifier
prisma/          schema.prisma (PostgreSQL), demo seed
scripts/         setup, set-key, ai:check
public/samples/  sample Zerodha, Upstox and Groww CSVs
```

Further reading in the repository: `README.md` (setup and demo script), `DECISIONS.md` (why things are the way they are) and `DEPLOY_CHECKLIST.md` (release checks and results).